

Solving Multiphysics and Multiscale Problems with AI-Augmented Simulations (CFD-PINN)

Part 2: Brief Introduction to PINNs

Jeff Gomes

(Twitter/X: @CoPhyMs)



XXIX ENMC
ENCONTRO NACIONAL DE MODELAGEM COMPUTACIONAL

XVII ECTM
ENCONTRO DE CIÊNCIA E TECNOLOGIA DE MATERIAS

Section Outline

- 1 Brief Introduction to Learning Theory of PINN
- 2 Simple Application: 1D Heat Equation Problem (Analytical Solution and PINN)

CONVERGENCE

About the solution of a PDE

- Given a *deep neural network* (DNN) u_θ parameterised to approximate the true PDE solution u , the primary theoretical question is whether the sequence of trained predictors converges to u .
- [De Ryck, Lanthaler & Mishra \(2021\)](#) proved that for networks with tanh activations, the approximation error vanishes as the network width goes to infinity.
- This only proves that a good network exists
→ It does not mean training will successfully get you there.

**De Ryck, Lanthaler & Mishra
(2021)**

$$\|u_\theta - u\| \rightarrow 0 \text{ as width} \rightarrow \infty$$

For tanh-activated networks, on suitable norms.

Empirical Risk and Risk

- PINN training is fundamentally an **Empirical Risk Minimization (ERM) problem**: it optimises over a finite set of collocation points, but evaluate performance over the entire continuum.
- **Empirical Risk**: The discrete loss over sampled training points (the optimisation target).
- **True (Population) Risk**: The integrated continuous loss over the full domain (the generalisation target).
- **Error Analysis**: Quantifies how effectively minimising empirical risk bounds the true risk.

Empirical risk

$$\hat{L}(\theta) = \frac{1}{N} \sum_{i=1}^N |r_{\theta}(x_i)|^2$$

Risk

$$L(\theta) = \int_{\Omega} |r_{\theta}(x)|^2 dx$$

Sources of Error

Optimisation error

The gap between the trained model and the best possible weights due to non-convex optimization.

Generalisation error

The gap between empirical risk on sampled points and true risk over the continuous domain.

Approximation error

How well the chosen network architecture (width, depth) can represent the true solution at all.

Gradient-based optimisers (Adam, L-BFGS) work well in practice, but finding a global minimum for a nonconvex PINN loss remains an open problem.

Global Error Bound

- [Kutyniok \(2022\)](#) shows that the global error between the trained network u_θ and the true solution u can be bounded by the sum of the three error sources.
- Schematically: total error $\lesssim$ [optimisation error + generalisation error + approximation error].
- Each term is controlled by a different lever: architecture (approximation), sampling strategy (generalisation), and optimiser choice (optimisation).
- This decomposition is the roadmap for essentially all PINN research on training reliability, including some advanced schemes.

Global error bound

$$\|u_\theta - u\| \lesssim \varepsilon_{\text{opt}} + \varepsilon_{\text{gen}} + \varepsilon_{\text{approx}}$$

Approximation and Generalisation Error

Approximation error

- De Ryck, Lanthaler & Mishra (2021): a tanh-activated network with only **two hidden layers** can approximate u with an explicit bound in a Sobolev space.

Generalisation error

- De Ryck & Mishra (2021): for Kolmogorov-type PDEs (e.g., heat equation, Black–Scholes), the generalisation error bound is **independent of the problem's dimension**.

Why this matters

For this class of PDEs, a small training error guarantees a small generalisation error. Thus, the method behaves independently of dimension. This is exactly [why PINNs excel](#) at high-dimensional problems, where traditional mesh-based solvers are crippled by the curse of dimensionality.

Information Bottleneck

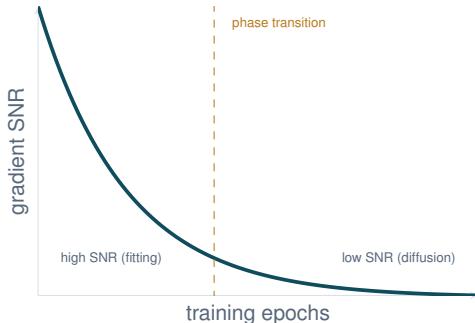
- **Information Bottleneck (IB)** theory offers an information-theoretic explanation of how neural networks learn and represent data internally.
- Each layer produces a transformed representation T of the input X , designed to: retain maximal information about the target Y , and discard all information not needed for that task.
- This trade-off is quantified via **mutual information**:
 - $I(X; T)$: dependence between input and layer state
 - $I(T; Y)$: dependence between layer state and output
- The optimal balance (compressing X while preserving all task-relevant information) defines the *information bottleneck*.

Two Phases of Learning

- **Fitting**: the network rapidly increases $I(T; Y)$, memorising the training signal.
- **Diffusion**: a slower phase where $I(X; T)$ decreases — most genuine generalisation happens here.

Signal-to-Noise Ratio (SNR) and the Phase Transition

- The transition between the fitting and diffusion phases is driven by the **signal-to-noise ratio (SNR)** of the gradients across training epochs.
- **High SNR**: gradients point consistently in a similar direction — the fitting phase.
- **Low SNR**: gradients become noisy and inconsistent between batches — the diffusion phase.



This figure was developed for general DNNs (Shwartz-Ziv & Tishby, 2017). The natural question is whether it also applies to PINNs.

Does this apply to PINNs?

Anagnostopoulos et al. (2024) extended this framework to PINNs, where multi-objective losses (PDE residuals + BC/IC) fundamentally alter the learning dynamics:

- **Aligned Gradients:** Loss terms act in unison $\Rightarrow$ deterministic, fitting-dominated regime.
- **Unaligned Gradients:** Loss terms conflict $\Rightarrow$ stochastic, diffusion-dominated regime, driven by multi-objective gradient noise.
- **Novel PINN Phase:** Low gradient variance yields a balanced regime ($\text{SNR} \approx 1$), positioned between pure fitting and pure diffusion.

This traces PINN training pathologies to *conflicting gradients* between loss terms, the central failure mode the rest of this lecture addresses.

Section Outline

- 1 Brief Introduction to Learning Theory of PINN
- 2 Simple Application: 1D Heat Equation Problem (Analytical Solution and PINN)

CONCEPTUAL

The 1+1-D Heat Equation

- We solve a 1D heat equation on $x \in [0, L]$, $t > 0$, with thermal diffusivity α .

$$u_t = \alpha u_{xx}, \quad x \in (0, L), \quad t > 0$$

- Dirichlet boundary conditions fix the temperature at both ends; an initial condition $u_0(x)$ sets the temperature profile at $t = 0$.

$$\text{BCs: } u(0, t) = u(L, t) = 0$$

$$\text{IC: } u(x, 0) = u_0(x)$$

- This problem is simple enough to solve by hand and ideal to compare the analytical solution against a PINN's output.

ANALYTICAL SOLUTION

Separation of Variables

- The problem is **separable**: seek solutions of the form $u(x, t) = X(x) T(t)$.
- Substituting into the PDE splits it into two ODEs, one in x and one in t , linked by a separation constant.
- The Dirichlet BCs force $X(x)$ into a discrete family of sine modes $\sin(n\pi x/L)$ (*i.e.*, an eigenvalue problem).
- This is functional analysis, in which we are projecting u_0 onto an orthonormal (Fourier sine) basis.

ANALYTICAL SOLUTION

The Full Solution

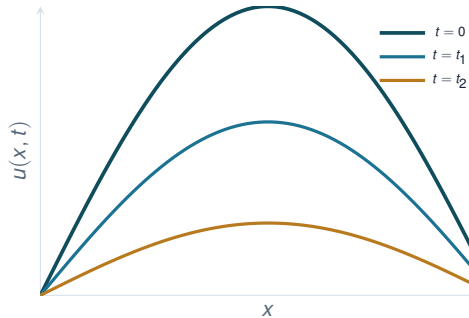
- Each mode decays exponentially in time, at a rate set by its spatial frequency: higher modes decay faster.
- The full solution is the Fourier sine series that matches the initial condition at $t = 0$.

$$u(x, t) = \sum_{n=1}^{\infty} b_n e^{-\alpha(n\pi/L)^2 t} \sin\left(\frac{n\pi x}{L}\right)$$

- As $t \rightarrow \infty$, every mode decays to zero, which is consistent with the homogeneous Dirichlet BCs draining all the heat out of the domain.

ANALYTICAL SOLUTION

The Full Solution (heat flux)



NUMERICAL SOLUTION

Solving the same problem with PINN

- We build a DNN $u_\theta(x, t)$ taking space and time as input, outputting the temperature field.
- Three residuals are combined into the total loss: the PDE residual (interior points), the boundary residual, and the initial-condition residual.
- Each residual is a mean-squared error over its own set of sampled (collocation) points.

The loss terms

$$L_{\text{PDE}} = \frac{1}{N_f} \sum_i |u_{\theta,t} - \alpha u_{\theta,xx}|^2_{(x_i, t_i)}$$

$$L_{\text{BC}} = \frac{1}{N_b} \sum_i |u_\theta(0, t_i)|^2 + |u_\theta(L, t_i)|^2$$

$$L_{\text{IC}} = \frac{1}{N_0} \sum_i |u_\theta(x_i, 0) - u_0(x_i)|^2$$

Solving the same problem with PINN

Step-by-Step

- 1 Exact solution function definition;
- 2 DNN definition (using PyTorch);
- 3 PDE class (residual via autograd);
- 4 Boundary/initial conditions class;
- 5 Assembling the whole PINN model;
- 6 Train model;
- 7 Test model against the exact solution

With the exact solution in hand, we can measure the PINN's true error directly, not just its training loss.

EXAMPLE 1

Cup of Coffee (Newton's cooling law)



[Link to GitHub](#)

Consider a hot cup of coffee at initial temperature, T_o and subject to the room temperature T_∞ . As expected from the Newton law of cooling, the coffee's temperature will change in time, *i.e.*, $T(t)$ and expressed as

$$\frac{dT}{dt} = -\frac{hA_s}{mC_p} (T - T_\infty),$$

where h , A_s , T_∞ and (mC_p) are the interphase heat transfer coefficient, surface area, environmental temperature and effective heat capacity, respectively. The analytical solution for this *Newton cooling* problem is

$$T(t) = T_\infty + (T_o - T_\infty) e^{-\tau^{-1}}, \text{ where } \tau = \frac{mC_p}{hA_s}.$$

Analytical Solution + PINN

